

P3185R0

A proposed direction for C++ Standard Networking based on IETF TAPS

Draft Proposal, 2024-03-13

This version:

<http://wg21.link/Pxxxx>

Author:

[Thomas Rodgers](#) (Woven By Toyota)

Audience:

SG4

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/rodgert/papers/blob/master/source/p3185r0.bs>

Abstract

A proposal align C++ Standard Networking with IETF's TAPS initiative

Table of Contents

- 1 Background**
- 2 What is Transport Services Application Programming Interface (TAPS)?**
 - 2.1 Why TAPS?
 - 2.2 API Model
 - 2.3 TAPS Concepts
 - 2.4 All data transfer is by Messages
 - 2.5 Flexible Connection establishment

3 Architecture

4 Open Question(s)

5 Conclusion

References

Informative References

§ 1. Background

In 2014, the C++ committee made a decision to "adopt existing practice" for a standard networking proposal, specifically to adopt a proposal based on the widely used [\[Asio\]](#) library. The NetworkingTS [\[netts\]](#) was the result of years of effort to meet that guidance from the direction setting groups within WG21. The resulting proposal didn't support various common requirements for many modern networking use cases, such as secure-by-default [\[p1861\]](#) networking. Somewhat independently a decision was made to adopt the Sender/Receiver [\[p2300\]](#) model for asynchrony. The NetworkingTS asynchrony model is not currently harmonized with the adopted direction of [\[p2300\]](#), which is something that [\[p2762\]](#) seeks to address. Recent SG4 discussion [\[p2762_kona\]](#) revived discussion around the desire to support connection-by-name which would necessitate the addition of a proposal for name resolution, parsing of string based interface paths, etc. The Kona discussion suggested that perhaps SG4 could look to the Internet Engineering Task Force's (IETF) Transport Services Application Programming Interface (TAPS) for the industry expert's current thinking on approaching this expanded feature set.

§ 2. What is Transport Services Application Programming Interface (TAPS)?

TAPS is a Standards Track initiative of the Internet Engineering Task Force (IETF) to describe a language agnostic abstract architecture for exposing transport protocol features to applications for network communication [\[TAPS_arch\]](#). The IETF has also published a language agnostic abstract interface [\[TAPS_iface\]](#) and implementation guidance [\[TAPS_impl\]](#).

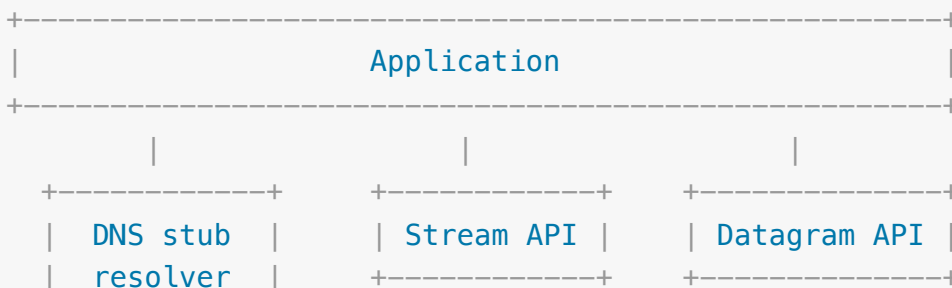
§ 2.1. Why TAPS?

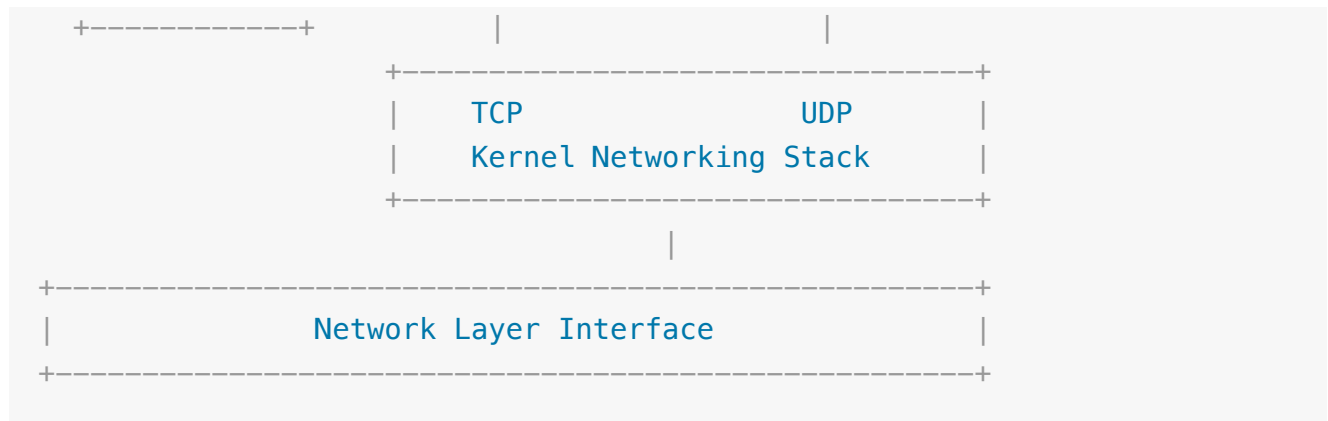
- TAPS is an IETF initiative to shape future networking API design based on lessons learned from building networking systems on Berkeley Sockets based designs.
- TAPS proposes a language agnostic API that covers areas that have been raised as shortcomings of the current Networking TS:
 - Named connections, including DNS resolution
 - Transport Layer Security
- TAPS seeks to address the inconsistent concepts and naming of existing networking APIs (often derived from Berkeley Sockets) where names and concepts often vary as a pure quirk of the history of "how we got to here" and based on the protocol being used.
- The TAPS architecture aims to redefine the interface between applications and transports in a way that allows the transport layer to evolve without fundamentally changing the contract with the application.
- The TAPS recommendations aim to make common use cases simple and consistent while still providing more fine grained control over the transport layer for those applications that require it.

§ 2.2. API Model

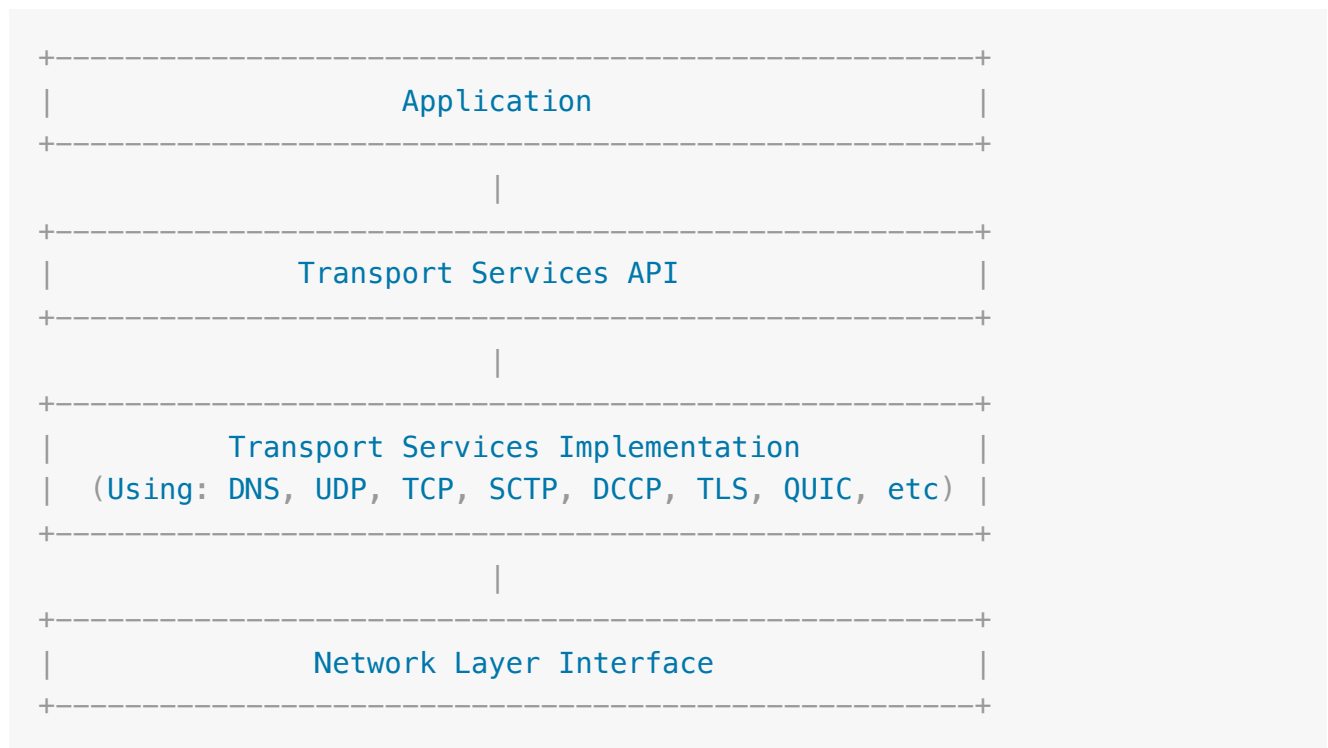
The traditional model, including that outlined in the Networking TS [\[netts\]](#), can be described as follows:

- Create connections and transfer data using a Sockets API.
- The Socket API provides the interface to implementations of the TCP and UDP protocols (typically provided by the system's kernel, or e.g. lwIP [\[lwip\]](#) in embedded systems).
- The TCP and UDP implementation send and receive data over the network layer interfaces.
- Sockets are bound directly to transport-layer and network-layer addresses, obtained via a separate resolution step, usually performed by a system-provided DNS stub resolver.





TAPS simplifies and modernizes this API structure to -



The API defines an interface for creating connections and transferring data, combining interfaces for multiple interaction patterns into a unified whole. The API offers generic support for protocols as protocol specific mappings for, e.g. TCP, UDP, etc. These protocol mappings are extensible to future protocol definitions, e.g. QUIC [\[rfc9000\]](#) or perhaps even the Binary Lexical Octet Ad-Hoc Transport [\[rfc3252\]](#).

By combining name resolution with connection establishment and data transfer, the API allows for more flexible implementations (a library QoI decision) to provide path and transport protocol agility on the application's behalf.

Key differences exist between the architecture of the TAPS design and the architecture of the Socket API: the TAPS API is asynchronous and event-driven; it uses messages to represent data transfer to

applications; and it describes how an application can resolve Endpoint Identifiers to use multiple IP addresses, multiple protocols, multiple paths, and provide multiple application streams.

§ 2.3. TAPS Concepts

Event driven API - Originally, the Socket API presented a blocking interface for establishing connections and transferring data. However, most modern applications interact with the network asynchronously. Various strategies are employed to implement this interface over the blocking sockets API, and the [\[netts\]](#) provides such an abstraction, while also exposing the blocking synchronous API of the underlying socket library.

In contrast to this model, all interactions with TAPS are expected to be asynchronous and the API is defined around an event driven model. For example, when an application uses the TAPS API to receive data, it issues an asynchronous call to receive new data (a value) from the Connection. When delivered data becomes available, this data (value) is delivered to the application using asynchronous events (`receiver::set_value`) that contain the data (a complete framed message value). Error handling is also asynchronous, resulting in asynchronous error events (`receiver::set_error`). In a design that meshes well with the sort of asynchronous programming model envisioned by [\[p2300\]](#).

Additional events are delivered asynchronously to the application regarding the lifetime of a connection and changes in network availability, which were not previously made explicit in the Socket API.

Separate from events, callbacks are also provided for interactions with the TAPS API that are not directly related to events on the network or network interfaces, typically for negotiating transport layer security.

§ 2.4. All data transfer is by Messages

The Socket API provides a message interface for datagram protocols like UDP, but provides an unstructured stream abstraction for TCP. While TCP has the ability to send and receive data as a byte-stream, most applications need to interpret structure within this byte-stream. For example, HTTP/1.1 uses character delimiters to segment messages over a byte-stream [\[RFC9112\]](#); TLS [\[RFC8446\]](#) record headers carry a version, content type, and length [\[RFC8446\]](#); and HTTP/2 uses frames to segment its headers and bodies [\[RFC9113\]](#). It is common for other protocols, e.g. [\[ZeroMQ\]](#) to use an embedded framing preamble that specifies the number of octets in the TCP stream which comprise a message.

TAPS represents data as messages, which more closely matches the way applications use the network. There are additional benefits to this message-oriented approach:

- Provide additional information to the protocol stack and application (e.g. a structured header type, separate from the message body).
- Ability to associate deadlines with messages, for applications that care about timing
- Ability to control reliability, which messages to retransmit when there is packet loss, and how best to make use of the data that arrived
- Ability to automatically assign messages and connections to underlying transport connections to utilize multi-streaming and pooled connections.

This message oriented approach is generally backwards compatible with existing usages where application code would have previously provided the framing of messages, while giving more information to the protocol stack in a consistent fashion that allows the application to adapt to transport level changes in a consistent manner. For protocols that inherently use streaming, or where applications use a non-standard or custom message framing, Framers bridge the gap between abstractions.

§ 2.5. Flexible Connection establishment

The Socket API for protocols like TCP and UDP is generally limited to connecting a single address over a single interface. It also presents a single stream to the application. Software layers built atop this abstraction frequently propagate this single-address single-stream model. The TAPS architecture provides:

- Handling multiple candidate endpoints, protocols, and paths
- Support for candidate protocol racing to select the most optimal stack in each situation
- Support multipath and multistreaming protocols
- State caching and application control over it.

A TAPS implementation is intended to be flexible at connection establishment time, considering many different options to select the most optimal connection that meets the requested criteria, with appropriate fallback strategies.

Information used in connection establishment (e.g. cryptographic resumption tokens, information about usability of certain protocols on the path, results of racing in previous connections) are cached by the implementation and applications have control over whether this information is used for a specific establishment, in order to allow trade offs between efficiency and link capability.

TAPS also allows for flexibility after connection reestablishment, for instance migration between multiple network-layer interfaces, allowing the application to react to interface changes, quality of service, etc.

§ 3. Architecture

Connections are described by properties:

- Flexible
- Fallback Handling

Connections are asynchronous:

- Sending
- Receiving

Message definition is flexible:

- Custom framing
 - [\[zeromq\]](#)
 - [\[rfc3252\]](#)

§ 4. Open Question(s)

- Should a C++ Networking Standard provide a high level interface, e.g. TAPS, or should it provide low level facilities, sufficient to build higher level application interfaces?
- Should a C++ Networking Standard admit the possibility to support more "interesting" transports, e.g. InfiniBand, RDMA, etc.?
 - If so, how does a more primitive networking standard's surface area support that? How does a TAPS based interface?
- TAPS has significant implementation complexity. Can the stdlib implementers adopt a proposal of this complexity?
 - I would argue it is an equivalent "lift" to what has already been proposed by the Networking TS along with enhancements to support Sender/Receiver, connection-by-name, and secure-by-default proposals.
 - A donated reference implementation would be helpful but not required, many of TAPS facilities are provided by existing libraries and operating systems. Standard libraries could choose to base their implementations on these libraries.

- Any donated implementation would require something akin to the standard libraries adopting [\[range_v3\]](#), [\[stdpar\]](#), [\[p2300\]](#), etc.
- If we adopt TAPS, we could finally foreclose on SG4 (circa 2012)'s most pressing concern, "What to do about IPv8"; with TAPS, this is simple, we approve an IPv8 transport and property set which is supported by the generic TransportProperties type. One could only dream of this job stability.

§ 5. Conclusion

Remember the [\[R101\]](#) - Should SG4 spend it's resources to design large scale changes to the NetworkingTS to support -

- A new async model
- Connection-by-name, endpoint resolvers, etc.
- Transport security
- etc.

Would such an effort yield a similarly cohesive result as the IETF's TAPS initiative?

§ References

§ Informative References

[Asio]

Chris Kohlhoff. *Asio C++ Library*. URL: <https://think-async.com/Asio>

[LWIP]

lwIP - A Lightweight TCP/IP stack. 2002-10-17. URL: <https://savannah.nongnu.org/projects/lwip>

[NETTS]

Working Draft, C++ Extensions for Networking. 2018-04-04. URL: <https://wg21.link/n4734>

[P1861]

Secure Networking in C++. 2020-05-11.

[P2300]

std::execution. 2023-04-21. URL: <https://wg21.link/p2300>

[P2762]

Sender/Receiver Interface For Networking. 2023-10-12. URL: <https://wg21.link/p2762>

[P2762_KONA]

Minutes - SG4 Nov 8, 2023 -- p2762: Presenting - Dietmar Kuhl. 2023-11-08. URL: <https://wiki.edg.com/bin/view/Wg21kona2023/SG4>

[R101]

R101 - Wikipedia. 1930-10-05. URL: <https://en.wikipedia.org/wiki/R101>

[RANGE_V3]

Eric Niebler. *range-v3*. URL: <https://github.com/ericniebler/range-v3>

[RFC3252]

Binary Lexical Octet Ad-hoc Transport. 2002-04-01. URL: <https://www.ietf.org/rfc/rfc3252.txt>

[RFC8446]

The Transport Layer Security (TLS) Protocol Version 1.3. 2018-08-01. URL: <https://datatracker.ietf.org/doc/html/rfc8446>

[RFC9000]

QUIC: A UDP-Based Multiplexed and Secure Transport. 2021-05-01. URL: <https://datatracker.ietf.org/doc/html/rfc9000>

[RFC9112]

HTTP/1.1. 2022-06-01. URL: <https://datatracker.ietf.org/doc/html/rfc9112>

[RFC9113]

HTTP/2. 2022-06-01. URL: <https://datatracker.ietf.org/doc/html/rfc9113>

[STDPAR]

Parallel STL. URL: <https://github.com/llvm/llvm-project/tree/main/pstl>

[TAPS_arch]

Architecture and Requirements for Transport Services. 2023-11-09. URL: <https://datatracker.ietf.org/doc/draft-ietf-taps-arch/>

[TAPS_iface]

Architecture and Requirements for Transport Services. 2023-11-09. URL: <https://datatracker.ietf.org/doc/draft-ietf-taps-interface/>

[TAPS_impl]

Architecture and Requirements for Transport Services. 2023-11-09. URL: <https://datatracker.ietf.org/doc/draft-ietf-taps-impl/>

[ZEROMQ]

ZeroMQ Message Transport Protocol. URL: <https://rfc.zeromq.org/spec/23/>